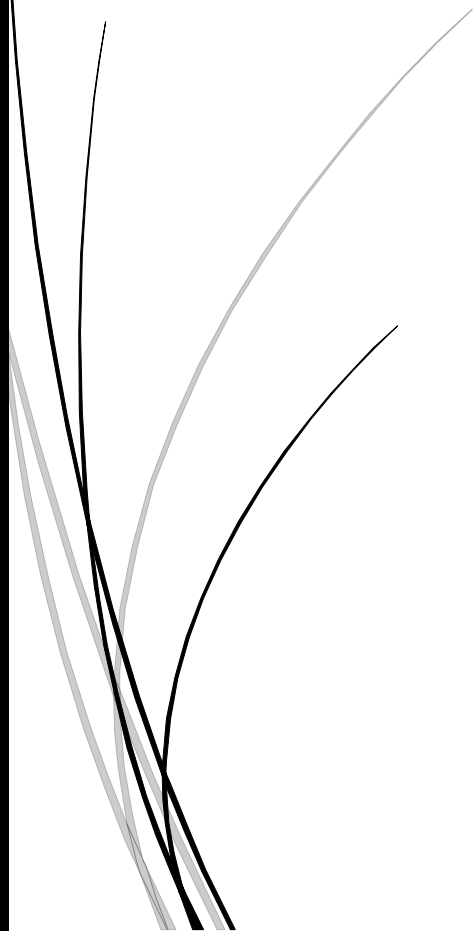
17-10-2026

Simulación de un sistema de líneas de espera M/M/1 mediante SimPy

Simulación



Wilson Patricio Palma Samaniego
UNIVERSIDAD NACIONAL DE LOJA

Simulación de un sistema de líneas de espera M/M/1 mediante SimPy

Guía de Actividades Práctico-Experimentales Nro. 003 — Práctica 03

Wilson Patricio Palma Samaniego

16 de octubre de 2026

Resumen

En esta práctica desarrollé una aplicación en Python para simular una línea de espera M/M/1 con la librería SimPy. El programa calcula las métricas teóricas del modelo, registra los tiempos de llegada, espera, servicio y salida de cada cliente, y compara los resultados de la simulación con los valores teóricos. También permite observar la evolución de la cola y la distribución de los tiempos de espera desde una interfaz local hecha con Streamlit. Trabajé con los tres escenarios de la guía (λ = 2.0, 2.5 y 2.9, con μ = 3.0) usando 10 000 clientes y la semilla 2026 en cada uno. Los resultados simulados quedaron cercanos a los teóricos y, como era de esperar, la cola y la espera crecieron bastante cuando la tasa de llegada se acercó a la tasa de servicio.

Índice

1 Datos generales 2
2 Introducción 2
3 Objetivos 3
3.1 Objetivo general 3
3.2 Objetivos específicos 3
4 Marco teórico 3
4.1 Líneas de espera y notación de Kendall 3
4.2 Características del modelo M/M/1 4
4.3 Métricas teóricas 4
4.4 Estabilidad y ley de Little 4
4.5 Simulación de eventos discretos 4
5 Materiales y herramientas 4
6 Metodología 5
6.1 Arquitectura del software 5
6.2 Modelo de simulación 7
6.3 Cálculo de las métricas simuladas 7
6.4 Intervalo de observación 7

7 Desarrollo e implementación 8
8 Resultados 8
8.1 Comparación entre resultados teóricos y simulados 8
8.2 Evolución del tamaño de la cola 9
8.3 Distribución de los tiempos de espera 10
8.4 Comparación entre escenarios 11
8.5 Evidencia de la interfaz 12
9 Análisis de resultados 13
10 Preguntas de control 15
11 Conclusiones 16
12 Recomendaciones 16
13 Código y material complementario 17
14 Declaración de uso de inteligencia artificial 17
15 Referencias 17

1 Datos generales

Universidad	Universidad Nacional de Loja
Facultad	Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables (FEIRNNR)
Carrera	Computación
Asignatura	Simulación
Ciclo	Quinto
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje	R1. Aplica análisis estadísticos de los datos de la simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica	03
Título	Simulación de un sistema de líneas de espera M/M/1 mediante SimPy
Docente	José O. Guamán Q.
Grupo	Grupo 2
Autor del informe	Wilson Patricio Palma Samaniego
Tipo de entrega	Informe individual
Fecha	16 de octubre de 2026
Lugar	Aula
Tiempo planificado	3 horas

2 Introducción

Las líneas de espera aparecen en situaciones como la atención en ventanillas, los centros de llamadas y los servidores informáticos. En estos casos, las personas o

solicitudes llegan en distintos momentos y deben esperar si el servidor está ocupado. Cuando llegan clientes casi al mismo ritmo con el que pueden ser atendidos, es común que aumenten la cola y el tiempo de espera [1], [2].

La teoría de colas permite calcular medidas como la cantidad promedio de clientes en espera, el tiempo que pasan en el sistema y la utilización del servidor [2], [3]. La simulación de eventos discretos complementa estos cálculos porque permite observar cómo cambia el sistema con el paso del tiempo [4].

En esta práctica se trabaja con el modelo M/M/1, que considera llegadas de Poisson, tiempos de servicio exponenciales y un solo servidor que atiende en orden de llegada. Como el modelo cuenta con fórmulas conocidas, es posible comparar sus valores teóricos con los resultados de una simulación.

El objetivo del trabajo es implementar el modelo en Python con SimPy, calcular sus métricas y compararlas en los tres escenarios de la guía. La aplicación incluye una interfaz local para ejecutar las simulaciones y consultar tablas y gráficas. El código de la aplicación se desarrolló de manera grupal; en este informe presento mi ejecución particular, mi tabla de métricas, mis gráficas y el análisis que realicé de forma individual.

3 Objetivos

3.1 Objetivo general

Desarrollar una aplicación en Python que simule un sistema M/M/1 con SimPy y permita comparar las métricas teóricas con las obtenidas en la simulación.

3.2 Objetivos específicos

- Implementar las llegadas, el servicio de un único servidor y la atención FIFO mediante SimPy [1].
- Calcular la utilización del servidor y las métricas teóricas ρ, L, L_q, W y W_q.
- Registrar los tiempos de llegada, espera, servicio y salida de los clientes.
- Comparar los resultados teóricos y simulados para los tres escenarios indicados.
- Observar cómo cambia la cola cuando la tasa de llegada se acerca a la tasa de servicio.

4 Marco teórico

4.1 Líneas de espera y notación de Kendall

La notación de Kendall $A/S/c$ resume las características de una línea de espera: A representa el proceso de llegadas, S la distribución del tiempo de servicio y c el número de servidores [3].

En el modelo M/M/1, la primera M indica que las llegadas siguen un proceso de Poisson, por lo que los tiempos entre llegadas tienen distribución exponencial. La

segunda M indica que los tiempos de servicio también siguen una distribución exponencial. El número 1 significa que hay un solo servidor [2], [3].

4.2 Características del modelo M/M/1

El modelo utilizado en esta práctica tiene las siguientes características:

- **Llegadas:** se generan de forma aleatoria con tasa media λ.
- **Servicio:** el tiempo de atención se genera con una distribución exponencial de tasa μ.
- **Servidor único:** solo se atiende a un cliente a la vez.
- **Disciplina FIFO:** el primero que llega es el primero en ser atendido.
- **Cola sin límite definido:** los clientes esperan hasta recibir el servicio.

Para analizar a cada cliente se registran su identificador, la llegada, el inicio del servicio, la espera en la cola, la duración del servicio y la salida del sistema [1].

4.3 Métricas teóricas

Las fórmulas siguientes son válidas para un sistema M/M/1 estable, es decir, cuando $\lambda < \mu$ [1]–[3].

Métrica	Símbolo	Fórmula
Utilización del servidor	ρ	$\rho = \lambda/\mu$
Promedio de clientes en el sistema	L	$L = \lambda/(\mu - \lambda)$
Promedio de clientes en la cola	L_q	$L_q = \lambda^2/[\mu(\mu - \lambda)]$
Tiempo promedio en el sistema	W	$W = 1/(\mu - \lambda)$
Tiempo promedio de espera en la cola	W_q	$W_q = \lambda/[\mu(\mu - \lambda)]$

La utilización se calcula como λ/μ. Un valor cercano a 1 indica que el servidor está ocupado gran parte del tiempo. No debe confundirse con μ/λ.

4.4 Estabilidad y ley de Little

Para que el modelo tenga un estado estacionario debe cumplirse $\lambda < \mu$, lo que equivale a $\rho < 1$. Si la tasa de llegada iguala o supera la tasa de servicio, las fórmulas anteriores dejan de tener sentido, porque la cola crecería sin límite [2], [3].

La ley de Little relaciona el promedio de clientes con el tiempo promedio que permanecen en el sistema: $L = \lambda W$. Para la cola, la relación correspondiente es $L_q = \lambda W_q$ [5].

4.5 Simulación de eventos discretos

En una simulación de eventos discretos, el estado del sistema cambia cuando ocurre un evento, como la llegada de un cliente o el final de un servicio [4]. SimPy gestiona el reloj de simulación y los procesos de llegada y atención. Así se pueden registrar los tiempos de los clientes y estimar las métricas a partir de lo que ocurre durante la ejecución [6].

5 Materiales y herramientas

Para desarrollar el proyecto se utilizan las siguientes herramientas, de acuerdo con la guía de la práctica [1]:

- **Python 3:** lenguaje de programación (la versión 3.12.10 en mi equipo).
- **SimPy:** simulación de eventos discretos.
- **NumPy:** generación de tiempos aleatorios.
- **Matplotlib:** elaboración de gráficas.
- **Streamlit:** interfaz web local.
- **unittest:** pruebas automatizadas incluida en Python.
- **Visual Studio Code:** editor de código.
- **Computador:** equipo con al menos dos núcleos de procesador y 4 GB de RAM, según la guía.

En mi ejecución trabajé sobre Windows, con un entorno virtual de Python y las versiones de librería registradas en el archivo requirements.txt del proyecto (SimPy 4.1.2, NumPy 2.5.4, Matplotlib 3.11.2 y Streamlit 1.66.0).

6 Metodología

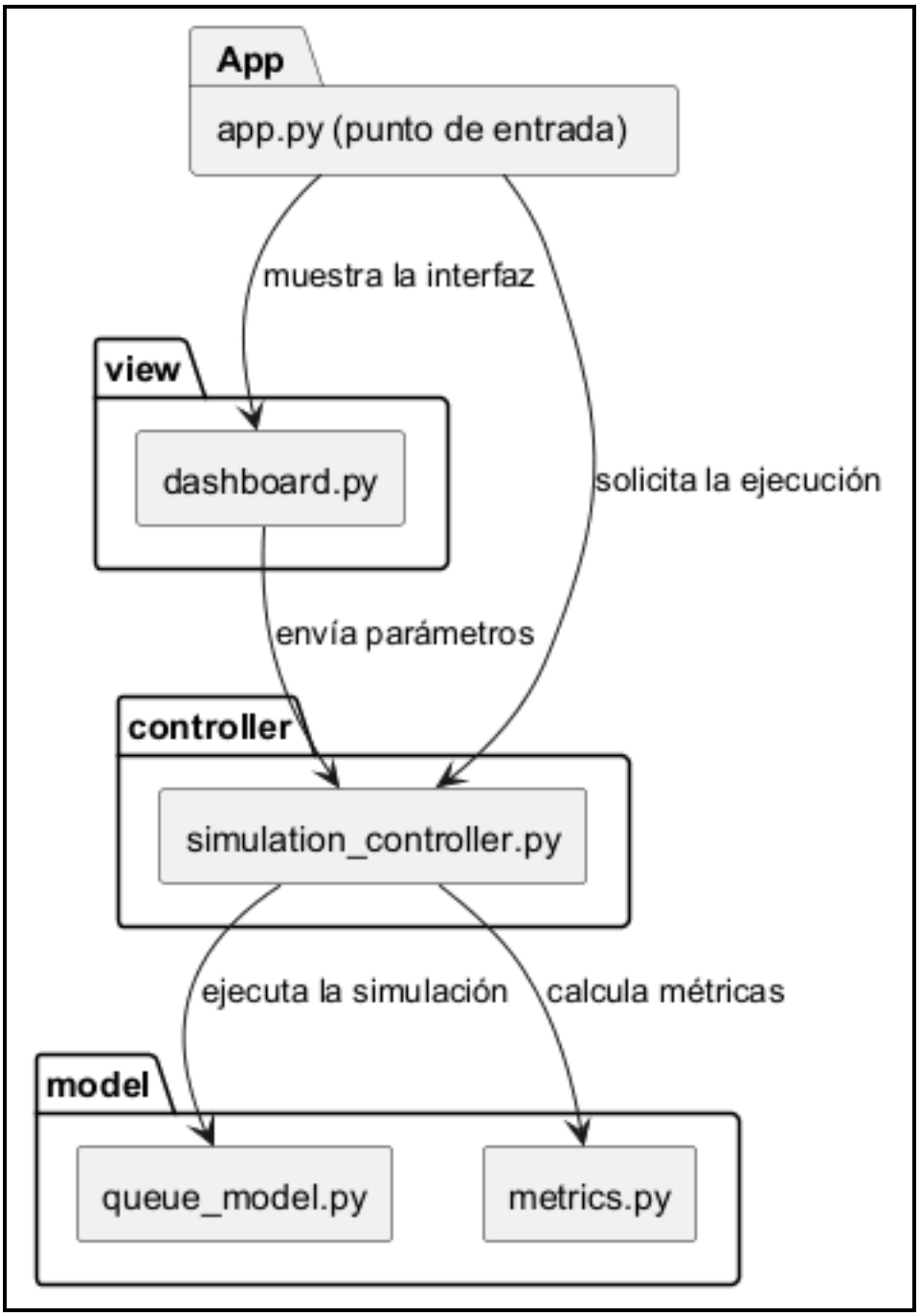
El trabajo se organizó en estas etapas: revisión del modelo teórico, diseño de la estructura del programa, implementación de la simulación, cálculo de las métricas, ejecución de los escenarios y comparación de los resultados.

6.1 Arquitectura del software

El programa se organizó siguiendo MVC (Modelo–Vista–Controlador) para separar la simulación de la interfaz:

- **Modelo:** contiene la lógica de la cola y los cálculos teóricos.
- **Controlador:** valida los parámetros, ejecuta los escenarios y organiza los resultados.
- **Vista:** presenta los controles, las tablas y las gráficas.
- **Punto de entrada:** inicia la aplicación y conecta sus componentes.

El siguiente diagrama resume la relación entre los módulos:



Archivo o carpeta	**Responsabilidad**
app.py	Inicia la aplicación y conecta la vista con el controlador.
model/queue_model.py	Gestiona las llegadas, el servidor y los tiempos de los clientes mediante SimPy.
model/metrics.py	Calcula las métricas teóricas y valida las tasas.
controller/simulation_controller.py	Define los escenarios, valida los parámetros y prepara la comparación.

Archivo o carpeta	Responsabilidad
view/dashboard.py	Presenta los controles, indicadores, tablas y gráficas.
tests/	Contiene las pruebas automatizadas.

6.2 Modelo de simulación

Para cada cliente se genera un tiempo entre llegadas exponencial, con media $1/\lambda$. Cuando llega, solicita el servidor. Si está ocupado, espera su turno; cuando queda libre, el cliente recibe atención y se genera su tiempo de servicio, con media $1/\mu$ [1], [6].

La lógica central del proceso puede resumirse así:

```
with server.request() as request:
    yield request
    service_start = env.now
    service_duration = rng.exponential(1.0 / mu_rate)
    yield env.timeout(service_duration)
```

El código completo también registra la llegada, la salida, la espera y los cambios en el tamaño de la cola. El tiempo de espera se calcula como service_start - arrival_time, mientras que el tiempo total en el sistema es departure_time - arrival_time.

6.3 Cálculo de las métricas simuladas

Las métricas teóricas se calculan con las fórmulas de la sección anterior y solo se presentan como valores estacionarios cuando $\lambda < \mu$.

Las métricas experimentales se calculan a partir de los eventos registrados:

- **Espera promedio en la cola:** promedio de los tiempos de espera de los clientes.
- **Tiempo promedio en el sistema:** promedio del tiempo entre la llegada y la salida.
- **Tamaño promedio de la cola:** área acumulada bajo la curva del número de clientes esperando, dividida por el tiempo observado.
- **Utilización simulada:** tiempo durante el cual el servidor estuvo ocupado, dividido por el tiempo total de observación.

El promedio de la cola se calcula por tiempo de observación, no promediando unas pocas lecturas instantáneas.

6.4 Intervalo de observación

La simulación genera una cantidad fija de clientes y continúa hasta que todos terminan su servicio. El tiempo de observación va desde 0 hasta T, donde T es el instante de salida del último cliente.

Esta decisión permite registrar a todos los clientes generados, aunque implica que la simulación empieza con el sistema vacío y termina cuando se atiende al último cliente. Por eso, los promedios obtenidos en una ejecución finita pueden diferir de los valores teóricos de estado estacionario.

7 Desarrollo e implementación

La aplicación permite ejecutar un escenario por separado o los tres en una misma operación. La cantidad de clientes y la semilla aleatoria se configuran desde la interfaz, mientras que las tasas de los escenarios de la guía se mantienen sin cambios [1].

Escenario	λ	μ	Característica	$\rho = \lambda/\mu$
1	2.0	3.0	Sistema estable	0.6667
2	2.5	3.0	Mayor utilización	0.8333
3	2.9	3.0	Cerca de la saturación	0.9667

Para que una ejecución pueda repetirse, el programa utiliza un generador aleatorio con semilla. Con los mismos parámetros, la misma semilla y la misma versión del código, los resultados son reproducibles; lo comprobé ejecutando dos veces la misma configuración y comparando las tablas resultantes.

El proyecto incluye 20 pruebas automatizadas hechas con unittest. Las ejecuté con el comando python -m unittest discover -s tests -v y todas pasaron (20 de 20). Las pruebas comprueban las fórmulas de las métricas teóricas, que la utilización sea λ/μ, el rechazo de tasas no válidas, la cantidad de clientes generada, la coherencia de los tiempos, el uso de un solo servidor, la reproducibilidad con la misma semilla y el funcionamiento de los tres escenarios.

8 Resultados

Los resultados que presento a continuación provienen de mi propia ejecución del programa en la interfaz y de la ejecución de los tres escenarios con **10 000 clientes** y **semilla 2026**, aplicando la misma semilla a cada escenario.

8.1 Comparación entre resultados teóricos y simulados

La siguiente tabla compara los valores del modelo con los obtenidos en la simulación. Las columnas marcadas como teóricas provienen de las fórmulas del modelo M/M/1 y las simuladas provienen de la ejecución del programa.

Escenario	λ	μ	ρ teórica	L teórica	L_q teórica	W teórica	W_q teórica	Utilización simulada	L_q simulado	W simulado	W_q simulado
1	2.0	3.0	0.6667	2.0000	1.3333	1.0000	0.6667	0.6683	1.2882	0.9825	0.6469
2	2.5	3.0	0.8333	5.0000	4.1667	2.0000	1.6667	0.8071	3.5563	1.7831	1.4533
3	2.9	3.0	0.9667	29.000	28.0333	10.000	9.6667	0.9753	24.4034	8.7683	8.4313

Los clientes atendidos fueron 10 000 en los tres escenarios y el intervalo de observación terminó en 5021.7271, 4086.5648 y 3454.9680 unidades de tiempo para los escenarios 1, 2 y 3, respectivamente. Como se ve, el escenario con mayor utilización tarda menos tiempo en atender la misma cantidad de clientes, porque las esperas son mayores y el servidor permanece ocupado casi todo el tiempo.

Las métricas simuladas quedaron cercanas a las teóricas, sobre todo en el escenario 1 y en la utilización de los tres escenarios. La diferencia más notoria

aparece en el escenario 3, donde la espera y la cola simuladas resultaron menores que las teóricas.

8.2 Evolución del tamaño de la cola

Las gráficas siguientes muestran cómo cambia la cantidad de clientes que esperan durante cada simulación. La línea punteada representa el valor teórico de L_q para comparar.

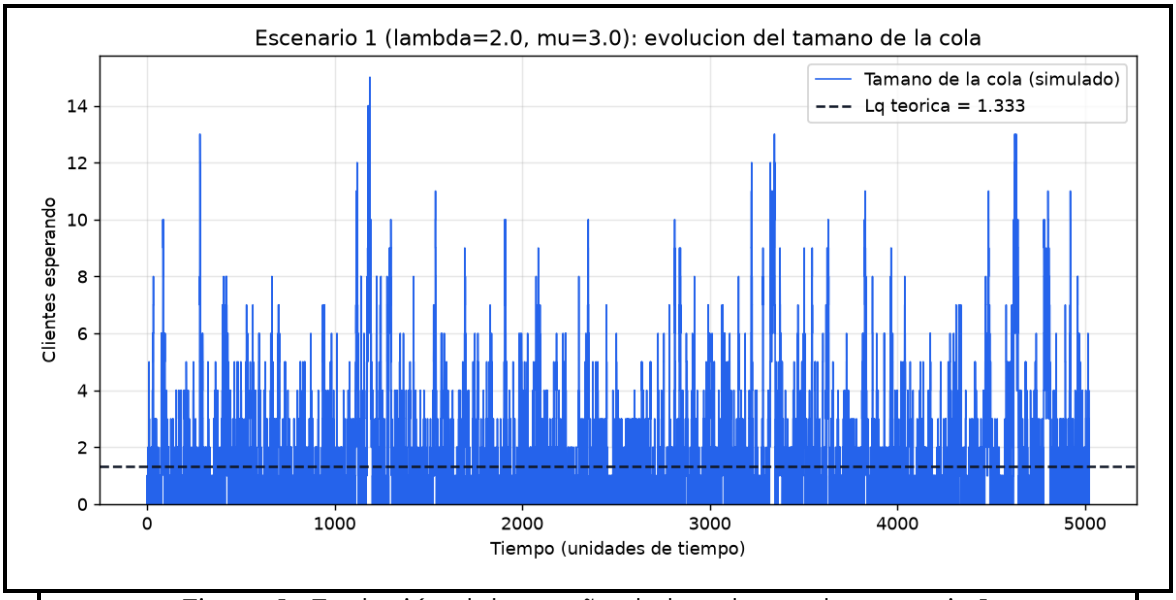


Figura 1 . Evolución del tamaño de la cola en el escenario 1.

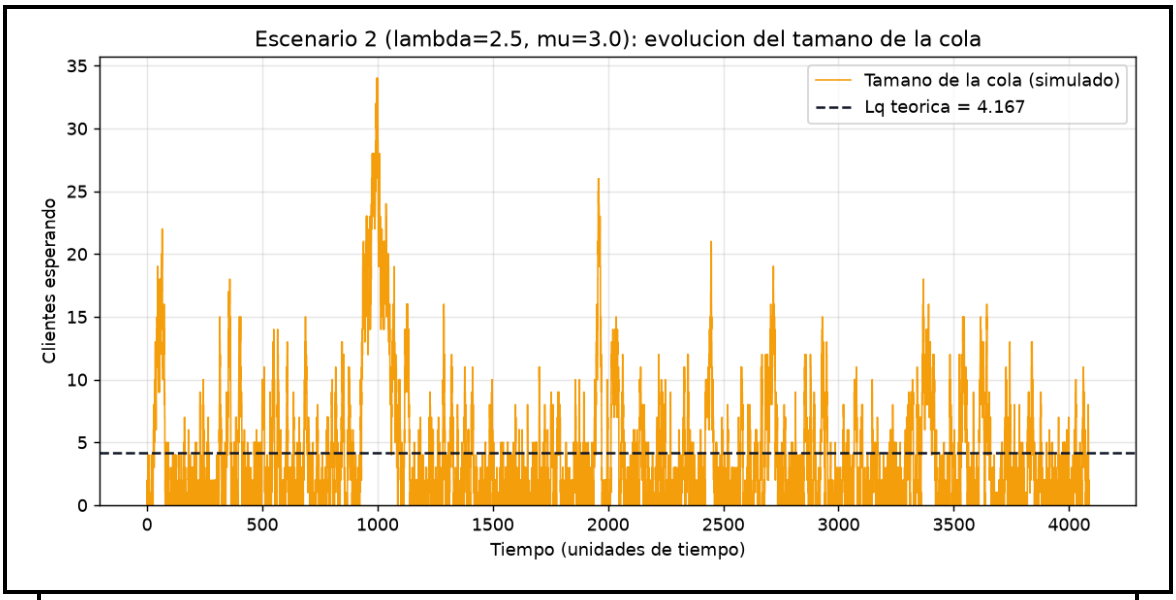


Figura 2 . Evolución del tamaño de la cola en el escenario 2.

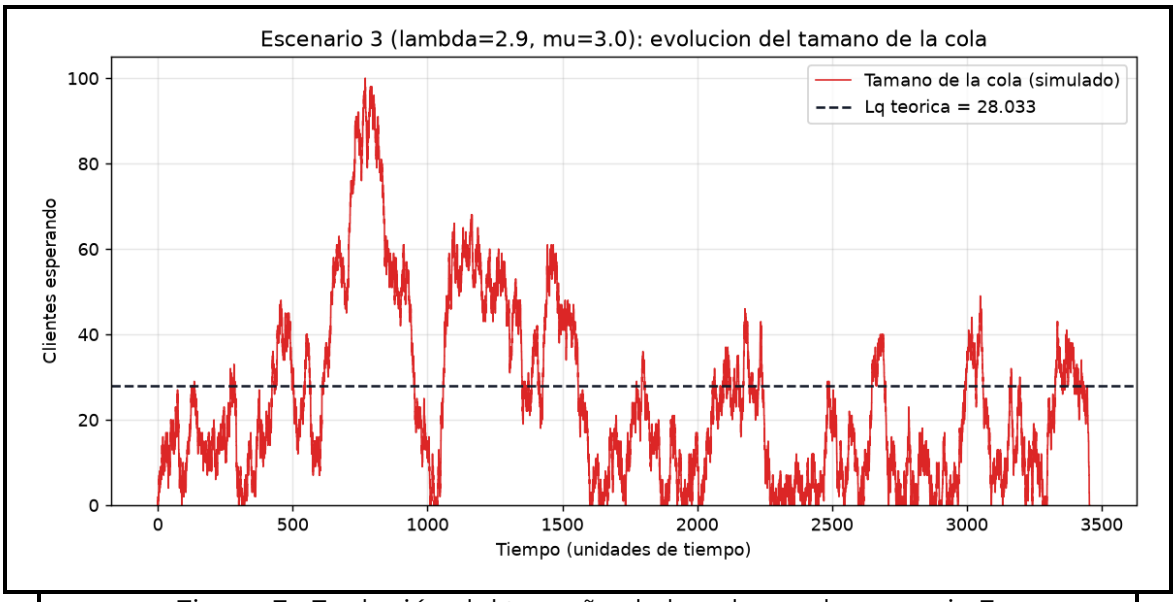


Figura 3 . Evolución del tamaño de la cola en el escenario 3.

En el escenario 1 la cola se mantiene baja y con muchas oscilaciones pequeñas. En el escenario 3 la cola alcanza valores mucho más altos y las subidas son más largas; incluso se observan tramos en los que casi no hay espera seguidos de picos grandes, lo que es típico cuando el sistema está al borde de la saturación.

8.3 Distribución de los tiempos de espera

Los histogramas permiten ver cómo se reparten los tiempos de espera de los 10 000 clientes. La línea punteada marca la W_q teórica y la línea sólida la W_q simulada.

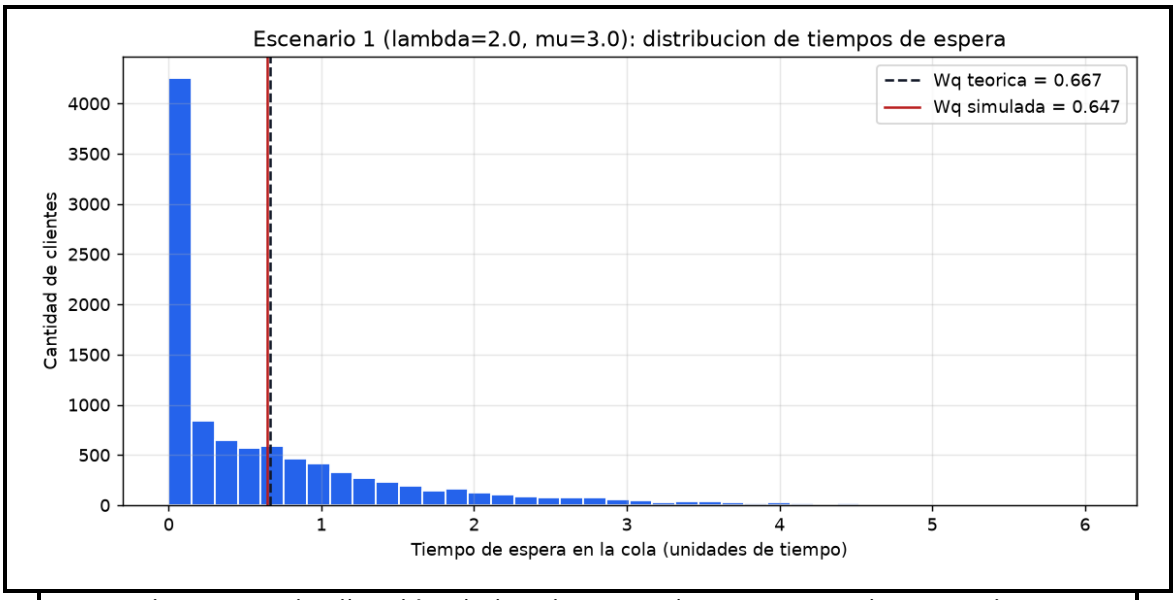


Figura 4 . Distribución de los tiempos de espera en el escenario 1.

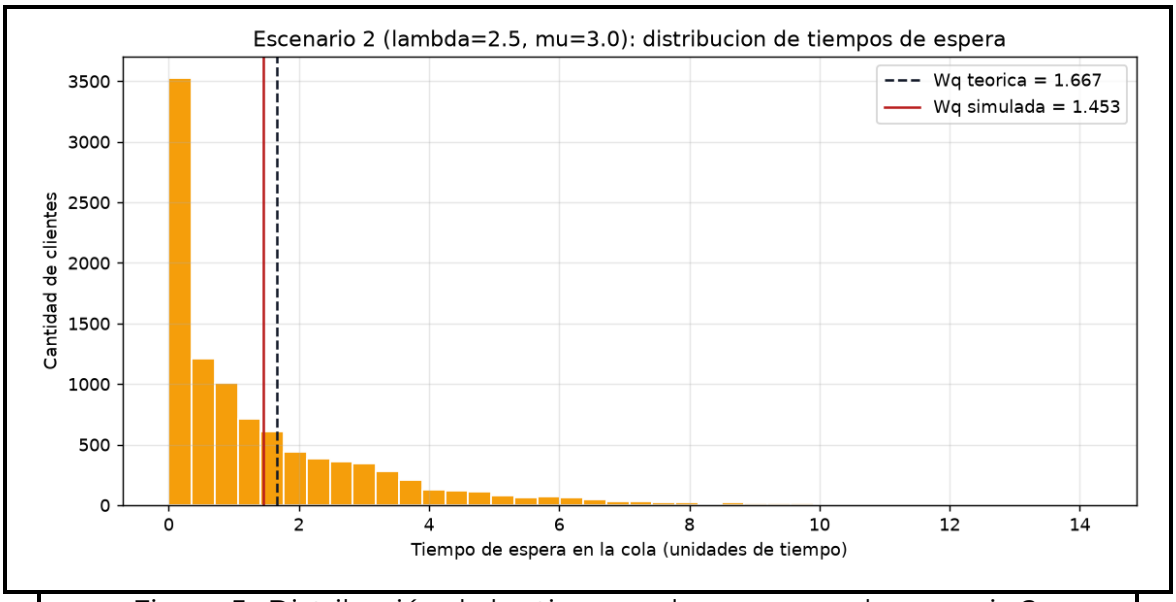


Figura 5 . Distribución de los tiempos de espera en el escenario 2.

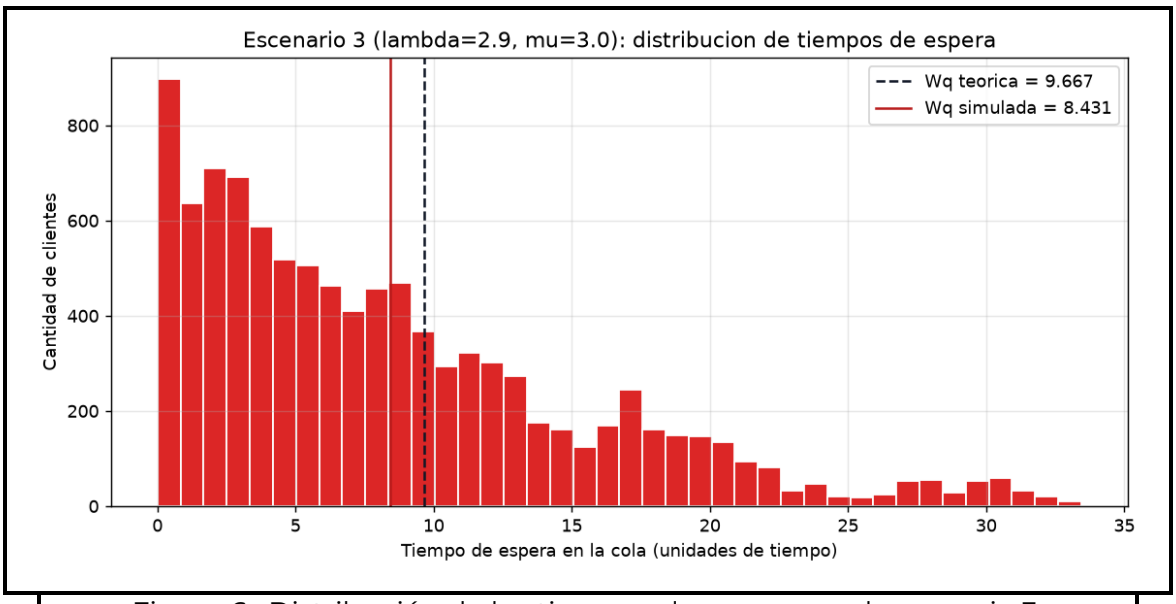


Figura 6 . Distribución de los tiempos de espera en el escenario 3.

En los tres casos la mayoría de los clientes espera poco tiempo, pero la cola de la distribución se alarga conforme aumenta λ. En el escenario 1 la espera máxima fue de 6.0333 unidades y en el escenario 3 llegó hasta 33.4405 unidades, es decir, más de cinco veces mayor.

8.4 Comparación entre escenarios

Estas gráficas comparan los valores teóricos con los simulados para el tiempo promedio de espera W_q y el tamaño promedio de la cola L_q.

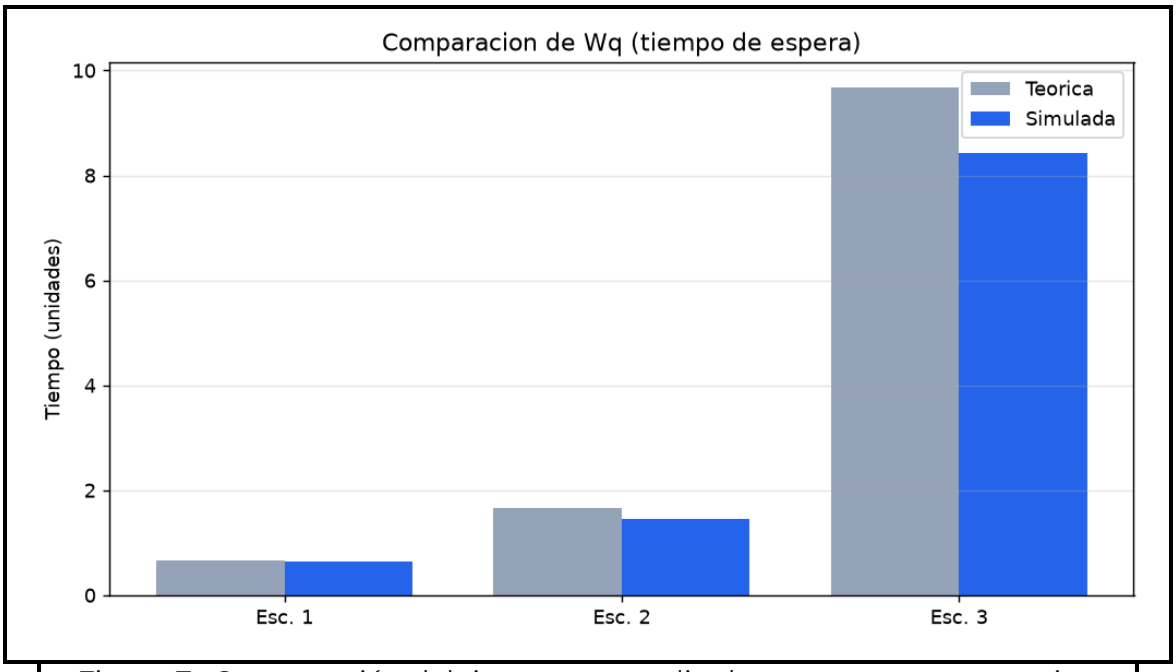


Figura 7 . Comparación del tiempo promedio de espera entre escenarios.

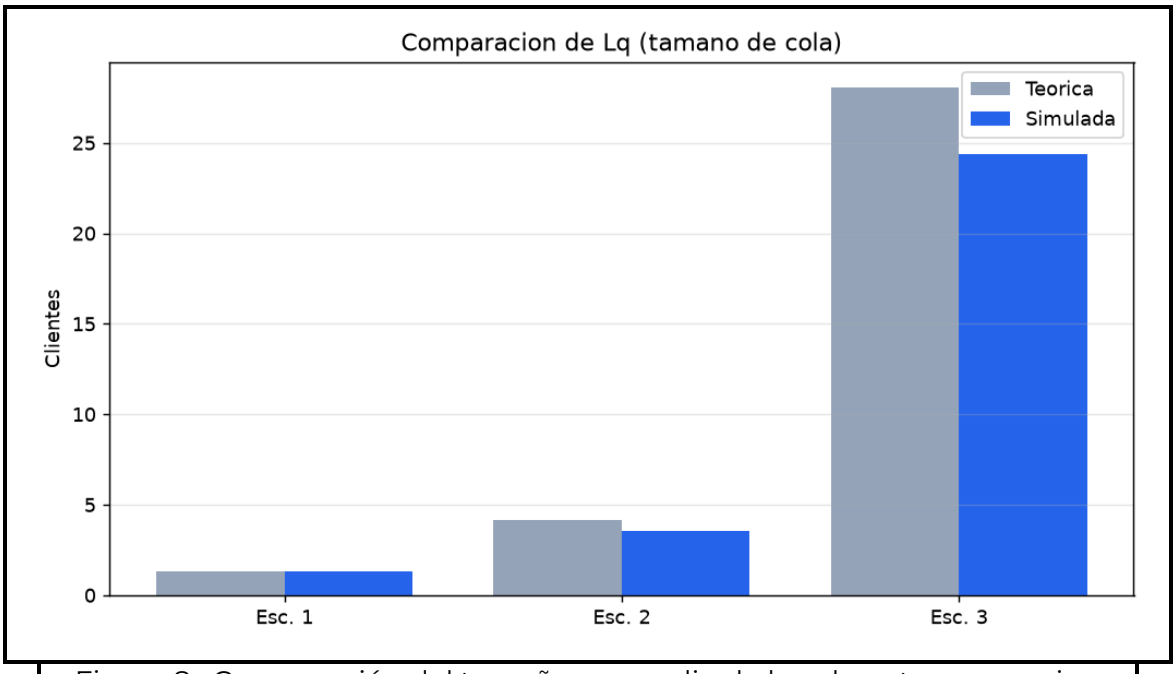


Figura 8 . Comparación del tamaño promedio de la cola entre escenarios.

Las barras de los escenarios 1 y 2 son muy parecidas entre teoría y simulación. En el escenario 3 la barra simulada queda por debajo de la teórica, además de estar muy por encima de las de los otros dos escenarios.

8.5 Evidencia de la interfaz

La siguiente captura muestra la interfaz de Streamlit después de ejecutar los tres escenarios en una misma corrida con 10 000 clientes y semilla 2026. En ella se observa el resumen de la comparación entre los valores teóricos y los simulados.



Figura 9 . Resultados de los tres escenarios en la interfaz de Streamlit.

9 Análisis de resultados

Lo primero que se nota en la tabla es que al aumentar λ crecen todas las métricas. La utilización teórica pasa de 0.6667 a 0.9667, y la espera promedio teórica sube de 0.6667 a 9.6667 unidades, es decir, unas catorce veces. Esto ocurre porque el servidor tiene cada vez menos tiempo libre para absorber las llegadas; pequeños

aumentos en la tasa de llegada producen diferencias grandes en la espera cuando ρ se acerca a 1 [2], [3].

En mi ejecución las métricas simuladas siguieron esa misma tendencia. La utilización simulada fue muy cercana a la teórica en los tres escenarios: 0.6683 frente a 0.6667, 0.8071 frente a 0.8333 y 0.9753 frente a 0.9667. Es decir, la proporción de tiempo que estuvo ocupado el servidor coincidió con el valor esperado con diferencias menores al 3 %.

La espera promedio simulada, en cambio, quedó algo por debajo de la teórica: 0.6469 frente a 0.6667 en el escenario 1 (una diferencia aproximada del 3 %), 1.4533 frente a 1.6667 en el escenario 2 (cerca del 13 %) y 8.4313 frente a 9.6667 en el escenario 3 (también cerca del 13 %). Una parte de esa diferencia se explica porque la simulación empieza con el sistema vacío: los primeros clientes encuentran el servidor libre y su espera es cero, así que arrastran el promedio hacia abajo. Esa parte inicial, llamada periodo de calentamiento, tiene más peso cuando el sistema tarda en estabilizarse, como sucede en el escenario 3. Las fórmulas, en cambio, describen el comportamiento de largo plazo, cuando el sistema ya lleva suficiente tiempo funcionando [1], [4].

Para revisar esto con más detalle comparé la ley de Little con mis resultados. Multiplicando la tasa de llegada por la espera promedio simulada se obtiene un valor que debería parecerse a la cola promedio simulada:

Escenario	$\lambda \cdot W_q$ simulado	L_q simulado	Diferencia relativa
1	1.2938	1.2882	0.44 %
2	3.6333	3.5563	2.12 %
3	24.4508	24.4034	0.19 %

Las diferencias son pequeñas, así que las métricas internas de la simulación son coherentes entre sí.

También quise ver cuánto influye el azar en los resultados, sobre todo en el escenario cercano a la saturación. Ejecuté el escenario 3 con 20 000 y 40 000 clientes y la espera promedio cambió a 6.6663 y 10.3735 unidades, es decir, mucho más de lo que se movieron los otros escenarios. Después repetí los tres escenarios diez veces cada uno, cambiando la semilla de 0 a 9, y obtuve lo siguiente:

Escenario	W_q teórica	W_q promedio de 10 réplicas	W_q mínima	W_q máxima
1	0.6667	0.6570	0.6235	0.6841
2	1.6667	1.5448	1.2805	2.0783
3	9.6667	9.4247	5.3838	15.2444

En el escenario 1 las diez réplicas dan valores muy parecidos entre sí y cercanos al teórico. En el escenario 2 la dispersión ya es visible y en el escenario 3 los valores van desde 5.3838 hasta 15.2444, un rango enorme alrededor del valor teórico de 9.6667. El promedio de las diez réplicas del escenario 3 (9.4247) sí queda bastante cerca de la teoría, pero una sola corrida puede alejarse bastante. Esto me parece el aprendizaje más importante de la práctica: cuando ρ se acerca a 1 no basta con

una sola ejecución; hay que repetir la simulación con varias semillas o usar más clientes para que los promedios sean confiables.

10 Preguntas de control

¿Qué representa el modelo M/M/1?

Una línea de espera con llegadas de Poisson, tiempos de servicio exponenciales y un único servidor que atiende en orden de llegada [2], [3].

¿Qué representan λ y μ?

λ es la tasa media de llegada de clientes y μ es la tasa media de servicio. Ambas se expresan en clientes por unidad de tiempo [1], [2].

¿Por qué debe cumplirse $\lambda < \mu$ para que el sistema sea estable?

Porque el servidor debe tener capacidad suficiente para atender, en promedio, a los clientes que llegan. Si las llegadas igualan o superan la capacidad de servicio, la cola crece sin límite y no existe un estado estacionario con métricas finitas [2], [3]. En los tres escenarios de la práctica se cumple esta condición, ya que λ llega como máximo a 2.9 y μ es 3.0.

¿Qué significa la utilización ρ?

Indica la proporción esperada de tiempo que el servidor está ocupado. Se calcula como $\rho = \lambda/\mu$ y, en un sistema estable, es menor que 1. En mi ejecución, la utilización medida con el tiempo de ocupación del servidor quedó muy cerca del valor teórico en los tres escenarios.

¿Cuál es la diferencia entre W y W_q?

W es el tiempo total desde que el cliente llega hasta que sale, es decir, la espera más el servicio. W_q cuenta únicamente el tiempo que espera antes de recibir el servicio. Por eso W siempre es mayor que W_q; en el escenario 3, por ejemplo, obtuve 8.7683 y 8.4313 respectivamente, y la diferencia de 0.3370 corresponde al tiempo promedio de servicio.

¿Cuál es la diferencia entre L y L_q?

L incluye a los clientes que esperan y al que está siendo atendido. L_q cuenta solo a quienes todavía esperan en la cola. La diferencia entre ambos es aproximadamente la utilización del servidor, porque en promedio hay ρ clientes en servicio.

¿Por qué se usa una distribución exponencial para los tiempos?

Porque el modelo M/M/1 supone llegadas de Poisson y tiempos de servicio exponenciales. Esa distribución representa los intervalos aleatorios del modelo y su media es justamente $1/\lambda$ para las llegadas y $1/\mu$ para el servicio [2], [3].

¿Qué sucede cuando λ se aproxima a μ?

La utilización se acerca a 1 y el servidor tiene menos tiempo libre. Por eso, la cola y las esperas aumentan considerablemente y también se vuelven más variables, como se vio en el escenario 3 [2].

¿Por qué pueden diferir los resultados simulados y los teóricos?

Porque la simulación usa una cantidad finita de valores aleatorios y parte de un sistema vacío. Las fórmulas describen promedios de estado estacionario, por lo que no se espera que una sola ejecución coincida exactamente con ellas [1], [4]. En mis resultados la diferencia fue pequeña en el escenario 1 y más grande en el escenario 3, que es el más cercano a la saturación.

¿Qué ocurriría si λ fuera mayor que μ?

El sistema sería inestable y la cola tendería a crecer indefinidamente. En ese caso las fórmulas de L, L_q, W y W_q no producirían valores finitos válidos, y el programa muestra un mensaje de error en lugar de calcular las métricas estacionarias [2], [3].

11 Conclusiones

- Se implementó y se ejecutó el modelo M/M/1 con SimPy, con llegadas aleatorias, tiempos de servicio exponenciales y un único servidor que atiende por orden de llegada.
- Las fórmulas teóricas coincidieron de forma razonable con la simulación. La utilización fue la métrica más estable: en mi ejecución se diferenció del valor teórico en menos del 3 % en los tres escenarios.
- La espera promedio simulada quedó ligeramente por debajo de la teórica, sobre todo en el escenario 3. Esto se explica por el periodo de calentamiento (la simulación empieza con el sistema vacío) y porque las fórmulas representan el estado estacionario.
- Al aumentar λ crecieron la utilización, la cola y la espera. En el escenario 3 la espera máxima llegó a 33.4405 unidades, más de cinco veces la del escenario 1.
- Repetir la simulación con varias semillas fue clave para interpretar los resultados. El promedio de diez réplicas del escenario 3 (9.4247) se acercó mucho al valor teórico (9.6667), mientras que una sola corrida pudo dar valores entre 5.3838 y 15.2444.
- La estructura MVC separa el modelo, el controlador y la interfaz, lo que facilita localizar errores y entender el funcionamiento del programa.
- Las 20 pruebas automatizadas del proyecto se ejecutaron sin errores, lo que da confianza en el cálculo de las métricas y en la coherencia de los tiempos registrados.

12 Recomendaciones

- Para los escenarios con ρ cercano a 1 conviene repetir la simulación con varias semillas y reportar el promedio, en lugar de confiar en una sola corrida.

- Sería útil añadir al programa un modo que ejecute varias réplicas y muestre el promedio y la desviación de las métricas; eso daría resultados más estables cerca de la saturación.
- También se podría analizar el efecto del periodo de calentamiento, descartando los primeros clientes de cada corrida, para ver si el promedio de espera se acerca más al valor teórico.
- Como trabajo futuro se puede ampliar el modelo a varios servidores o a una cola con capacidad limitada, para comparar con lo aprendido en este sistema de un solo servidor.

13 Código y material complementario

- **Repositorio del código:** https://github.com/wilsonpalma/ciclo5_simulacion/tree/main/semana3

14 Declaración de uso de inteligencia artificial

Como autor del informe declaro el uso de una herramienta de inteligencia artificial durante el desarrollo de la práctica:

- **Herramienta:** OpenCode, asistente de programación basado en inteligencia artificial.
- **Uso:** apoyo para organizar los módulos del programa, escribir partes del código, preparar las pruebas, revisar errores y redactar el README.
- **Verificación personal:** ejecuté el programa en mi equipo, obtuve las métricas que aparecen en este informe, generé las gráficas y comprobé el funcionamiento de la aplicación y de las pruebas antes de presentarlas.
- **Prompt utilizado:** el prompt principal que se proporcionó a OpenCode está documentado en el README del repositorio.

15 Referencias

[1] J. O. Guamán Q., "Guía de Actividades Práctico-Experimentales Nro. 003: Simulación de un sistema de líneas de espera M/M/1 mediante SimPy," Universidad Nacional de Loja, Loja, Ecuador, oct. 2026.

[2] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley-Interscience, 1975.

[3] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 4th ed. Hoboken, NJ, USA: John Wiley & Sons, 2008.

[4] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[5] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[6] Team SimPy, "SimPy: Discrete event simulation for Python," *SimPy Documentation*. Disponible en: https://simpy.readthedocs.io/en/latest/. Consultado en octubre de 2026.

[7] NumPy Developers, “NumPy reference,” *NumPy Documentation*. Disponible en: https://numpy.org/doc/stable/. Consultado en octubre de 2026.

[8] Matplotlib Development Team, “Matplotlib documentation.” Disponible en: https://matplotlib.org/stable/. Consultado en octubre de 2026.

[9] Streamlit Inc., “Streamlit documentation.” Disponible en: https://docs.streamlit.io/. Consultado en octubre de 2026.